

Chapter 43

A First Look at Deep Learning

Compute a single neuron by hand and recognize it as logistic regression in disguise · explain why one neuron can't solve every problem, and watch a network fail and then succeed on a real example · compute a full forward pass through a small network by hand and match it exactly in PyTorch

Approved chapter · v1

Contents

Chapter 43. A First Look at Deep Learning

Why this matters

In plain English

43.1 A neuron is logistic regression

43.2 Why one neuron isn't enough

43.3 A full forward pass, by hand

43.4 Backpropagation: the chain rule, automated

43.5 A network on tabular data

43.6 A first image classifier

43.7 Transfer learning

43.8 When to reach for deep learning

Common mistakes and how to spot them

In the real world: the deep learning pitch that needed a baseline

Tools

The project: a tabular network and an image classifier, evaluated honestly

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 43. A First Look at Deep Learning

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: compute a single neuron by hand and recognize it as logistic regression in disguise · explain why one neuron can't solve every problem, and watch a network fail and then succeed on a real example · compute a full forward pass through a small network by hand and match it exactly in PyTorch · understand backpropagation as the chain rule applied automatically, and check it against a hand-nudged gradient · train a small neural network on tabular data and compare it honestly with logistic regression and gradient boosting · understand what a convolution does, and train a small image classifier · walk through transfer learning end to end, and see when it helps and when it doesn't.

Before you start: Chapter 35 (gradients, gradient descent, the sigmoid), Chapter 36 (splits and pipelines), Chapter 37 (logistic regression, bias and variance), Chapter 41 (word embeddings, as a preview of learned representations).

Time needed: 8–12 hours over one to two weeks. This chapter runs entirely on a CPU in minutes; no GPU is needed.

Tools: Python 3 with PyTorch and scikit-learn (both free).

Practice data: Riverstone's customer accounts (Chapter 37, for the tabular network) and scikit-learn's built-in handwritten-digit images (1,797 small images, no download required, for the image classifier and transfer-learning walkthrough). Every number in this chapter was calculated, and every output shown is real.

Why this matters

Deep learning gets credited with everything from language models to self-driving cars, and that reputation makes it tempting to reach for a neural network on every problem. This chapter's honest position, backed by its own numbers, is narrower and more useful: **a neural network is a flexible way to stack the ideas from Chapters 35 and 37 into deeper, more expressive models**, and it earns its complexity on some problems (images, text, audio, huge datasets) and not on others (Riverstone's tabular churn data, where gradient boosting still wins).

Three things you'll be able to do by the end:

- Explain what a neural network actually computes, without hand-waving, because you'll have done the arithmetic yourself.
- Train one in PyTorch, on a table of numbers and on a set of images, and evaluate it the same honest way Chapters 37 and 39 taught.

- Answer the interview question "when would you use deep learning instead of gradient boosting?" with a real comparison behind you, not a guess.

In plain English

Think back to Chapter 37's **logistic regression: a weighted sum of features, squeezed through a sigmoid, giving a probability**. That whole calculation is what people call a **neuron**. One neuron is exactly one logistic regression.

A **layer** is a group of neurons, each looking at the same inputs and computing its own weighted sum. A **network** stacks layers: the outputs of one layer become the inputs to the next. Stacking matters because a single neuron can only draw one straight-line boundary through the data, and some real patterns need more than a straight line, no matter how you tilt it.

Training a network is exactly Chapter 35's gradient descent, applied to many more parameters at once. The one new piece of machinery is **backpropagation**: an efficient way to compute the gradient of every parameter in every layer, using the chain rule from calculus, applied automatically by the library so you never do it by hand in practice. This chapter has you do it by hand *once*, on a tiny example, purely so the automation stops feeling like magic.

43.1 A neuron is logistic regression

```
import math

import numpy as np

x = np.array([16, 5.03]) # Sharma Hardware: orders, revenue (Ch 35/37 style feature)
w = np.array([0.05, 0.9])
b = -1.2

z = w @ x + b
a = 1 / (1 + math.exp(-z))
print(f"z = w . x + b = {w[0]} x {x[0]} + {w[1]} x {x[1]} + ({b}) = {z:.3f}")
print(f"sigmoid(z) = {a:.3f}")
print("this is exactly Chapter 37's logistic regression, one input row at a time")
```

```
z = w . x + b = 0.05 x 16.0 + 0.9 x 5.03 + (-1.2) = 4.127
sigmoid(z) = 0.984
this is exactly Chapter 37's logistic regression, one input row at a time
```

Reading it. Nothing here is new. A neuron computes a weighted sum plus a bias, then applies an **activation function**, here the sigmoid, to squash the result into a useful range. Chapter 37's logistic regression is a single neuron with a sigmoid activation; deep learning starts by giving that neuron company.

A second activation function: ReLU

Sigmoid saturates: for large positive or negative inputs, its output barely changes, which slows learning (the gradient shrinks toward zero). Modern networks mostly use **ReLU** (rectified linear unit) in their hidden layers instead: it passes positive values through unchanged and zeroes out negative ones.

```
def relu(z):
    return max(0.0, z)

for z in [-2, -0.1, 0, 0.5, 3]:
    print(f"relu({z:>5}) = {relu(z)}    sigmoid({z:>5}) = {1 / (1 + math.exp(-z)):.3f}")
```

```
relu(  -2) = 0.0    sigmoid(  -2) = 0.119
relu( -0.1) = 0.0    sigmoid( -0.1) = 0.475
relu(   0) = 0.0    sigmoid(   0) = 0.500
relu(  0.5) = 0.5    sigmoid(  0.5) = 0.622
relu(   3) = 3      sigmoid(   3) = 0.953
```

Reading it. ReLU is almost insultingly simple, and that simplicity is the point: its gradient is either exactly 0 or exactly 1, never vanishingly small, which lets gradients flow through many stacked layers without shrinking to nothing. Sigmoid is still used, but usually only at a network's *output*, where you specifically want a 0–1 probability.

43.2 Why one neuron isn't enough

A single neuron draws one straight decision boundary. Some patterns can't be separated by any straight line, however you set the weights. The classic example is **XOR** — "true if exactly one of two conditions holds":

```
import torch

torch.manual_seed(43)

X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])
print("XOR: a customer gets a discount if exactly one of two conditions holds")
for row, target in zip(X_xor.tolist(), y_xor.tolist()):
    print(f"  inputs {row} -> target {target[0]:.0f}")
```

```
XOR: a customer gets a discount if exactly one of two conditions holds
  inputs [0.0, 0.0] -> target 0
  inputs [0.0, 1.0] -> target 1
  inputs [1.0, 0.0] -> target 1
  inputs [1.0, 1.0] -> target 0
```

Plot those four points and try to draw one straight line that puts the two 1s on one side and the two 0s on the other. It can't be done. Train a single neuron on it anyway and watch what happens:

```
import torch.nn as nn

single_neuron = nn.Sequential(nn.Linear(2, 1), nn.Sigmoid())
optimizer = torch.optim.SGD(single_neuron.parameters(), lr=0.5)
loss_fn = nn.BCELoss()

for step in range(2000):
    optimizer.zero_grad()
    prediction = single_neuron(X_xor)
    loss = loss_fn(prediction, y_xor)
    loss.backward()
    optimizer.step()

print(f"after 2000 steps, loss = {loss.item():.4f}")
print("predictions:", prediction.detach().numpy().round(3).ravel())
print("targets:      ", y_xor.numpy().ravel())
```

```
after 2000 steps, loss = 0.6931
predictions: [0.5 0.5 0.5 0.5]
targets:      [0. 1. 1. 0.]
```

Reading it. After 2,000 training steps, the loss has settled at 0.6931 — which is *exactly* $\ln(2)$, the loss of predicting 50% for everything (Chapter 35's base-rate benchmark). The single neuron gave up and learned nothing at all, because there is genuinely no straight line, and therefore no setting of its weights, that solves this problem. This isn't a training failure to be fixed with more steps or a better learning rate; it's a **structural limitation** of a single neuron.

Now add one hidden layer:

```
torch.manual_seed(43)

two_layer = nn.Sequential(nn.Linear(2, 4), nn.Tanh(), nn.Linear(4, 1), nn.Sigmoid())
optimizer2 = torch.optim.SGD(two_layer.parameters(), lr=0.5)

for step in range(3000):
    optimizer2.zero_grad()
    prediction2 = two_layer(X_xor)
    loss2 = loss_fn(prediction2, y_xor)
    loss2.backward()
    optimizer2.step()

print(f"after 3000 steps, loss = {loss2.item():.5f}")
print("predictions:", prediction2.detach().numpy().round(3).ravel())
print(f"total parameters: {sum(p.numel() for p in two_layer.parameters())}")
```

```
after 3000 steps, loss = 0.00097
predictions: [0.001 0.999 0.999 0.001]
total parameters: 17
```

Reading it. With a hidden layer of just 4 neurons between the input and the output, the network solves XOR almost perfectly (predictions of 0.001, 0.999, 0.999, 0.001 against targets of 0, 1, 1, 0). The hidden layer lets the network bend its decision boundary instead of being stuck with a straight line — each hidden neuron draws its own line, and the output layer combines them into a shape a single line

never could. **This is the entire reason depth exists:** stacking layers with nonlinear activations between them lets a network represent patterns a single layer structurally cannot, no matter how it's trained.

43.3 A full forward pass, by hand

Take the smallest possible multi-layer network — 2 inputs, 2 hidden neurons (tanh activation), 1 output (sigmoid) — and push one example through it, in PyTorch and by hand, to see that nothing is hidden:

```
x_tiny = torch.tensor([1.0, 0.5])
W1 = torch.tensor([[0.3, -0.2], [0.4, 0.1]]) # 2 hidden units x 2 inputs
b1 = torch.tensor([0.1, -0.1])
W2 = torch.tensor([[0.5, -0.6]]) # 1 output x 2 hidden units
b2 = torch.tensor([0.2])

z1 = W1 @ x_tiny + b1
a1 = torch.tanh(z1)
z2 = W2 @ a1 + b2
a2 = torch.sigmoid(z2)

print(f"hidden layer: z1 = {z1.tolist()} a1 = tanh(z1) = {a1.tolist()}")
print(f"output layer: z2 = {z2.item():.4f} a2 = sigmoid(z2) = {a2.item():.4f}")
```

```
hidden layer: z1 = [0.30000001192092896, 0.3500000238418579] a1 = tanh(z1) =
[0.29131263494491577, 0.33637556433677673]
output layer: z2 = 0.1438 a2 = sigmoid(z2) = 0.5359
```

```
z1_0 = 0.3 * 1.0 + (-0.2) * 0.5 + 0.1
z1_1 = 0.4 * 1.0 + 0.1 * 0.5 + (-0.1)
a1_0, a1_1 = math.tanh(z1_0), math.tanh(z1_1)
z2_hand = 0.5 * a1_0 + (-0.6) * a1_1 + 0.2
a2_hand = 1 / (1 + math.exp(-z2_hand))
print(f"by hand: z1 = [{z1_0:.2f}, {z1_1:.2f}] a1 = [{a1_0:.4f}, {a1_1:.4f}]")
print(f"by hand: z2 = {z2_hand:.4f} a2 = {a2_hand:.4f}")
print("matches PyTorch exactly:", abs(a2_hand - a2.item()) < 1e-9)
```

```
by hand: z1 = [0.30, 0.35] a1 = [0.2913, 0.3364]
by hand: z2 = 0.1438 a2 = 0.5359
matches PyTorch exactly: True
```

Reading it. Every digit matches PyTorch's own numbers, because they're the same calculation. `W1` holds one row of weights per hidden neuron; `W1 @ x` computes both hidden neurons' weighted sums in a single matrix multiplication (Chapter 35). This is what "forward pass" means: multiply, add a bias, activate, and repeat for each layer, ending at a single output.

43.4 Backpropagation: the chain rule, automated

Training needs the gradient of the loss with respect to *every* weight in *every* layer. Computing that by hand for a network with millions of parameters is unthinkable; **backpropagation** computes it efficiently by applying the calculus chain rule backward through the network, layer by layer, reusing work as it goes. PyTorch's `autograd` does this for you — `.backward()` is backpropagation, one line, however many layers exist.

To trust it, check it against a slower, cruder method Chapter 35 already used for a similar purpose: nudge one weight by a tiny amount and measure how much the loss changes.

```
W1g = W1.clone().requires_grad_()
b1g = b1.clone().requires_grad_()
W2g = W2.clone().requires_grad_()
b2g = b2.clone().requires_grad_()
target_tiny = torch.tensor([1.0])

def tiny_forward(W1, b1, W2, b2):
    return torch.sigmoid(W2 @ torch.tanh(W1 @ x_tiny + b1) + b2)

prediction_tiny = tiny_forward(W1g, b1g, W2g, b2g)
loss_tiny = nn.functional.binary_cross_entropy(prediction_tiny, target_tiny)
loss_tiny.backward()
print(f"loss = {loss_tiny.item():.4f}")
print(f"autograd dL/dW2 = {W2g.grad.tolist()}")

epsilon = 1e-4
with torch.no_grad():
    W2_plus = W2.clone()
    W2_plus[0, 0] += epsilon
    loss_plus = nn.functional.binary_cross_entropy(
        tiny_forward(W1, b1, W2_plus, b2), target_tiny
    )
    W2_minus = W2.clone()
    W2_minus[0, 0] -= epsilon
    loss_minus = nn.functional.binary_cross_entropy(
        tiny_forward(W1, b1, W2_minus, b2), target_tiny
    )
    numerical_grad = (loss_plus - loss_minus) / (2 * epsilon)
print(f"numerical dL/dW2[0,0] (nudge and measure) = {numerical_grad.item():.4f}")
```

```
loss = 0.6238
autograd dL/dW2 = [[-0.1351993978023529, -0.15611329674720764]]
numerical dL/dW2[0,0] (nudge and measure) = -0.1350
```

Reading it. Autograd's gradient (-0.1352) and the hand-nudged numerical gradient (-0.1350) agree to three decimal places, with the small remaining difference explained entirely by the nudge (`epsilon`) not being infinitesimally small. **This is what it means for backpropagation to be "just" the chain rule, automated:** it computes the exact same answer a patient, tedious nudge-and-measure process would give, only far faster and exactly rather than approximately. You'll never write backpropagation by hand in practice; you now know precisely what `.backward()` is doing when you call it.

43.5 A network on tabular data

Chapter 37 built churn models with logistic regression, random forests, and gradient boosting on Riverstone's accounts. Here's the same data and the same pipeline, feeding a small neural network instead:

```

import warnings

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["rep_id"] = accounts["rep_id"].astype(str)
CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

rest, test_acc = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
train_acc, valid_acc = train_test_split(
    rest, test_size=0.25, random_state=37, stratify=rest["churned_2025"]
)

prepare = ColumnTransformer(
    [
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
        (
            "num",
            Pipeline(
                [
                    ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                    ("scale", StandardScaler()),
                ]
            ),
            NUMS,
        ),
    ],
)

X_train = prepare.fit_transform(train_acc[CATS + NUMS]).astype("float32")
X_valid = prepare.transform(valid_acc[CATS + NUMS]).astype("float32")
y_train = train_acc["churned_2025"].to_numpy().astype("float32")
y_valid = valid_acc["churned_2025"].to_numpy().astype("float32")
print(
    f"{X_train.shape[0]:,} training accounts, {X_train.shape[1]} features "
    f"(same pipeline as Chapter 37)"
)

```

3,000 training accounts, 25 features (same pipeline as Chapter 37)

```

from sklearn.metrics import log_loss, roc_auc_score

torch.manual_seed(43)
Xtr_t = torch.tensor(X_train)
ytr_t = torch.tensor(y_train).unsqueeze(1)
Xva_t = torch.tensor(X_valid)

network = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
optimizer3 = torch.optim.Adam(network.parameters(), lr=0.01)
loss_fn3 = (
    nn.BCEWithLogitsLoss()
) # sigmoid + log loss combined, for numerical stability

for epoch in range(200):
    optimizer3.zero_grad()
    logits = network(Xtr_t)
    loss3 = loss_fn3(logits, ytr_t)
    loss3.backward()
    optimizer3.step()
    if epoch % 40 == 0:
        with torch.no_grad():
            valid_pred = torch.sigmoid(network(Xva_t)).numpy().ravel()
            print(
                f"epoch {epoch:>3}: training loss {loss3.item():.4f}   "
                f"validation AUC {roc_auc_score(y_valid, valid_pred):.3f}"
            )

with torch.no_grad():
    final_pred = torch.sigmoid(network(Xva_t)).numpy().ravel()
print(
    f"\nfinal: AUC {roc_auc_score(y_valid, final_pred):.3f}   "
    f"log loss {log_loss(y_valid, final_pred):.4f}   "
    f"parameters {sum(p.numel() for p in network.parameters()):,}"
)

```

```

epoch   0: training loss 0.6994   validation AUC 0.542
epoch  40: training loss 0.2510   validation AUC 0.745
epoch  80: training loss 0.2404   validation AUC 0.763
epoch 120: training loss 0.2341   validation AUC 0.766
epoch 160: training loss 0.2262   validation AUC 0.771

```

```

final: AUC 0.773   log loss 0.2812   parameters 561

```

Reading it. Training loss falls steadily and validation AUC climbs from a coin toss (0.542) to 0.771 within 160 epochs — this is Chapter 35's gradient descent, now optimizing 561 parameters spread across three layers instead of one. `BCEWithLogitsLoss` combines the sigmoid and the log loss into a single, more numerically stable step, which is why the network's last layer has no activation function of its own (`nn.Linear(8, 1)` with nothing after it): the loss function applies the sigmoid internally.

The honest comparison

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression

logit_model = LogisticRegression(max_iter=2000).fit(X_train, y_train)
logit_pred = logit_model.predict_proba(X_valid)[: , 1]

boosting_model = HistGradientBoostingClassifier(
    learning_rate=0.05, max_depth=3, min_samples_leaf=40, max_iter=300, random_state=37
).fit(X_train, y_train)
boosting_pred = boosting_model.predict_proba(X_valid)[: , 1]

print(f"{'model':<24} AUC      log loss   parameters")
print(
    f"{'logistic regression':<24} {roc_auc_score(y_valid, logit_pred):.3f}   "
    f"{'log_loss(y_valid, logit_pred):.4f}      {X_train.shape[1] + 1}"
)
print(
    f"{'small neural network':<24} {roc_auc_score(y_valid, final_pred):.3f}   "
    f"{'log_loss(y_valid, final_pred):.4f}      561"
)
print(
    f"{'gradient boosting':<24} {roc_auc_score(y_valid, boosting_pred):.3f}   "
    f"{'log_loss(y_valid, boosting_pred):.4f}      (hundreds of trees)"
)
```

model	AUC	log loss	parameters
logistic regression	0.764	0.2833	26
small neural network	0.773	0.2812	561
gradient boosting	0.788	0.2731	(hundreds of trees)

Reading it, plainly. The small neural network (AUC 0.773) narrowly beats logistic regression (0.764) and loses to tuned gradient boosting (0.788), while using twenty times more parameters than logistic regression and needing far more code, tuning, and training time than either. This matches Chapter 37's finding exactly and extends it: **on clean, modest-sized tabular data, deep learning does not have an automatic advantage** over the simpler methods from earlier chapters. It can match or slightly beat linear models; it does not reliably beat tuned gradient boosting, which remains the strongest default for tables of numbers. Where deep learning's advantage becomes overwhelming is exactly where the *simpler methods have no equivalent at all*: raw images, audio, and text at scale, which is where the next two sections go.

43.6 A first image classifier

From pixels to a prediction

An image is a grid of numbers — pixel brightness values. A **convolution** slides a small grid of learned weights (a **kernel** or **filter**) across the image, computing a weighted sum at each position, producing a new grid that highlights whatever pattern the kernel has learned to detect (an edge, a curve, a corner).

Stack several convolutional layers and the network builds up from simple patterns (edges) to more complex ones (loops, strokes) automatically, purely by gradient descent — nobody designs the kernels by hand.

```
from sklearn.datasets import load_digits

digits = load_digits()
images = digits.images.astype("float32") / 16.0 # pixel values 0-16 -> 0-1
labels = digits.target
print(
    f"{len(images):,} images, each {images.shape[1]}x{images.shape[2]} pixels, "
    f"digits {sorted(set(labels))}"
)
print("one image (rounded to 1 decimal):")
print(np.round(images[0], 1))
print(f"label: {labels[0]}")
```

```
1,797 images, each 8x8 pixels, digits [np.int64(0), np.int64(1), np.int64(2), np.int64(3),
np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9)]
one image (rounded to 1 decimal):
[[0.  0.  0.3 0.8 0.6 0.1 0.  0. ]
 [0.  0.  0.8 0.9 0.6 0.9 0.3 0. ]
 [0.  0.2 0.9 0.1 0.  0.7 0.5 0. ]
 [0.  0.2 0.8 0.  0.  0.5 0.5 0. ]
 [0.  0.3 0.5 0.  0.  0.6 0.5 0. ]
 [0.  0.2 0.7 0.  0.1 0.8 0.4 0. ]
 [0.  0.1 0.9 0.3 0.6 0.8 0.  0. ]
 [0.  0.  0.4 0.8 0.6 0.  0.  0. ]]
label: 0
```

Here's what one convolution step looks like, using a kernel *designed* by hand (real networks learn kernels like this from data instead):

```
edge_kernel = np.array([[ -1, -1, -1], [0, 0, 0], [1, 1, 1]]) # detects horizontal edges
patch = images[0][2:5, 2:5]
response = (patch * edge_kernel).sum()
print("a 3x3 patch of the image:")
print(np.round(patch, 2))
print(f"\nconvolving with a horizontal-edge kernel: {response:.2f}")
print(
    "(a real convolutional layer learns kernels like this one from data, "
    "instead of them being hand-designed)"
)
```

```
a 3x3 patch of the image:
[[0.94 0.12 0. ]
 [0.75 0.  0. ]
 [0.5  0.  0. ]]

convolving with a horizontal-edge kernel: -0.56
(a real convolutional layer learns kernels like this one from data, instead of them being hand-
designed)
```

Reading it. The horizontal-edge kernel (negative weights on top, positive on the bottom) produces a large response where the image transitions from dark to light going downward, and a small or negative response on flat regions. A trained convolutional layer has dozens of such kernels, each

tuned by gradient descent to detect whatever pattern reduces the loss most, not to detect "horizontal edges" specifically — that's simply an example humans can recognize and verify by eye.

Training a small CNN

```
class SmallCNN(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc1 = nn.Linear(16 * 2 * 2, 32)
        self.fc2 = nn.Linear(32, n_classes)

    def features(self, x):
        x = self.pool(torch.relu(self.conv1(x))) # 8x8 -> 4x4
        x = self.pool(torch.relu(self.conv2(x))) # 4x4 -> 2x2
        return torch.relu(self.fc1(x.flatten(1)))

    def forward(self, x):
        return self.fc2(self.features(x))

base_mask = labels < 5
X_base, y_base = images[base_mask], labels[base_mask]
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_base, y_base, test_size=0.25, random_state=43, stratify=y_base
)

def to_tensors(images_, labels_):
    return torch.tensor(images_).unsqueeze(1), torch.tensor(labels_).long()

Xb_train_t, yb_train_t = to_tensors(Xb_train, yb_train)
Xb_test_t, yb_test_t = to_tensors(Xb_test, yb_test)

torch.manual_seed(43)
base_model = SmallCNN(n_classes=5)
optimizer4 = torch.optim.Adam(base_model.parameters(), lr=0.01)
loss_fn4 = nn.CrossEntropyLoss()

for epoch in range(60):
    optimizer4.zero_grad()
    output = base_model(Xb_train_t)
    loss4 = loss_fn4(output, yb_train_t)
    loss4.backward()
    optimizer4.step()

with torch.no_grad():
    base_accuracy = (base_model(Xb_test_t).argmax(1) == yb_test_t).float().mean().item()
    print(f"digits 0-4: {len(Xb_train):,} training images, {len(Xb_test):,} test images")
    print(
        f"test accuracy: {base_accuracy:.1%} "
        f"parameters: {sum(p.numel() for p in base_model.parameters()):,}"
    )
```

```
digits 0-4: 675 training images, 226 test images
test accuracy: 98.7% parameters: 3,493
```

Reading it. With 675 training images and a network of only 3,493 parameters — tiny by deep-learning standards — the model correctly classifies 98.7% of held-out images of the digits 0–4. Two convolutional layers followed by max-pooling (which shrinks the grid, keeping only the strongest response in each small region) turn an 8×8 grid into a compact set of features that a small final layer can classify accurately.

43.7 Transfer learning

The idea

Training a network from nothing needs a reasonable amount of data. **Transfer learning** instead starts from a network already trained on a *related* task, keeps its early layers (which have learned general-purpose features), and retrain only the final layers on the new, smaller task. The reasoning: a convolutional layer that learned to detect edges and strokes for one set of digits should recognize similar edges and strokes in a different set of digits, without needing to relearn them.

```
new_mask = labels >= 5
X_new, y_new = images[new_mask], labels[new_mask] - 5 # relabel 5-9 as 0-4
Xn_train, Xn_test, yn_train, yn_test = train_test_split(
    X_new, y_new, test_size=0.25, random_state=43, stratify=y_new
)
Xn_test_t, yn_test_t = to_tensors(Xn_test, yn_test)

rng = np.random.default_rng(43)

def small_training_set(n_per_class):
    chosen = []
    for digit_class in range(5):
        candidates = np.where(yn_train == digit_class)[0]
        chosen.extend(rng.choice(candidates, n_per_class, replace=False))
    return to_tensors(Xn_train[chosen], yn_train[chosen])

print("new task: recognize digits 5-9 (reabeled 0-4), with only a handful of examples")
```

```
new task: recognize digits 5-9 (reabeled 0-4), with only a handful of examples
```

```

import copy

def train_transfer(n_per_class, epochs=80):
    Xs, ys = small_training_set(n_per_class)
    model = copy.deepcopy(base_model)
    for param in model.conv1.parameters():
        param.requires_grad = False
    for param in model.conv2.parameters():
        param.requires_grad = False
    model.fc1 = nn.Linear(16 * 2 * 2, 32) # fresh classifier head for the new task
    model.fc2 = nn.Linear(32, 5)
    trainable = filter(lambda p: p.requires_grad, model.parameters())
    optimizer = torch.optim.Adam(trainable, lr=0.01)
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xn_test_t).argmax(1) == yn_test_t).float().mean().item()

def train_scratch(n_per_class, epochs=80):
    Xs, ys = small_training_set(n_per_class)
    model = SmallCNN(n_classes=5)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xn_test_t).argmax(1) == yn_test_t).float().mean().item()

print(
    f"{'examples/class':>15}    {'transfer (frozen features)':>27}    {'from scratch':>13}"
)
for n in [3, 5, 10, 15, 30]:
    transfer_acc = train_transfer(n)
    scratch_acc = train_scratch(n)
    print(f"{n:>15}    {transfer_acc:>27.1%}    {scratch_acc:>13.1%}")

```

examples/class	transfer (frozen features)	from scratch
3	84.4%	75.4%
5	85.7%	94.2%
10	90.6%	92.4%
15	93.8%	92.4%
30	92.9%	98.2%

Reading it, honestly. There's no clean, uniform win for either approach here. Transfer learning helps most at the very smallest sample size — with only 3 examples per class, frozen features reach 84.4% while training from scratch manages only 75.4%, because 15 total images genuinely isn't enough to learn useful convolutional filters from nothing. But as the training set grows, from-scratch training catches up and eventually overtakes: by 30 examples per class, training everything from scratch reaches 98.2% against transfer learning's 92.9%.

Why doesn't transfer learning dominate here, the way its reputation suggests it should? Because the *base* task (recognizing digits 0–4, from only 675 training images) is itself small and narrow. Transfer learning's dramatic real-world wins come from base models trained on **enormous, general** datasets — a network trained on millions of diverse photographs learns edge and texture detectors far richer than anything 675 tiny digit images can teach, and *that* richness is what transfers well to a new, small task. A base model trained on a small, narrow dataset has correspondingly narrow, less transferable features. **The lesson isn't "transfer learning doesn't work"; it's "transfer learning is only as good as what the base model actually learned,"** and that has to be checked, not assumed, exactly like every other technique in this book.

Watch out: "pretrained" is doing a lot of work in that sentence. In practice, most transfer learning starts from a model trained on a dataset far larger and more general than anything in this chapter (millions of natural images, or, for text, billions of words — Chapter 54). The small, honest example here demonstrates the *mechanism* correctly; don't extrapolate its exact numbers to production transfer learning, where the base model's training data is usually the deciding factor.

43.8 When to reach for deep learning

Situation	Reach for
Tabular data (a table of customer, transaction, or account features)	Gradient boosting or logistic regression first (Chapter 37); a network rarely beats a well-tuned boosted model here
Images	A convolutional network, ideally starting from a strong pretrained base (transfer learning), not from scratch
Text at scale	Modern pretrained language models (Chapter 54) rather than a network trained from nothing
A small dataset (hundreds to low thousands of rows)	Simpler methods generally need less data to generalize well; deep learning shines with much more data
You need to explain individual predictions to a regulator or manager	Simpler models (Chapters 37, 39) are far easier to interpret than a deep network
Millions of rows, complex interactions, and no interpretability requirement	Deep learning becomes competitive with, and sometimes ahead of, boosted trees

The pattern across this book has been consistent: try the simplest thing that could work, measure it honestly, and only add complexity that earns its place with evidence (Chapter 36's baselines, Chapter 37's algorithm comparisons). Deep learning is not an exception to that discipline; it's one more tool to test against it.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reaching for a neural network on tabular data by default	More code, more tuning, no better result than boosting	Compare against Chapter 37's methods before committing
No hidden layer, or no activation function between layers	The network behaves like plain linear/logistic regression	Add at least one hidden layer with a nonlinear activation (ReLU, tanh)
Sigmoid in every hidden layer of a deep network	Training stalls; gradients shrink toward zero	Use ReLU (or a variant) in hidden layers; save sigmoid for the output
Forgetting <code>optimizer.zero_grad()</code>	Gradients accumulate across steps; loss behaves erratically	Zero gradients at the start of every training step
Applying sigmoid and then <code>BCELoss</code> mismatched with logits	Numerical instability, especially with <code>BCEWithLogitsLoss</code>	Use <code>BCEWithLogitsLoss</code> on raw outputs, without an extra sigmoid
Assuming transfer learning always helps	No improvement, or a worse result, than training from scratch	Check what the base model was actually trained on and how much data you have
Judging a deep learning result without a baseline	"95% accuracy!" with no comparison	Always compare with the algorithms from Chapter 37 (and a base-rate check from Chapter 35)
Training and evaluating on the same data	Impressive numbers that don't hold up	Chapter 36's splits and Chapter 39's evaluation habits apply unchanged

In the real world: the deep learning pitch that needed a baseline

In October 2026, a vendor pitches Riverstone a "deep learning churn prediction platform," with a demo showing 91% accuracy. Vikram, having learned the pattern from Chapter 37's AutoML story, asks Meera to check it the same way.

Meera doesn't need to build a competing neural network to answer the question. She already has this chapter's own comparison: on Riverstone's real churn data, a small neural network reaches an AUC of 0.773, gradient boosting reaches 0.788, and — the number that actually matters here — Riverstone's 9.7% churn rate means predicting "no churn" for everyone is already 90.3% accurate. The vendor's headline 91% accuracy is barely above that baseline, and accuracy alone, as Chapter 39 established, is the wrong number to trust on an imbalanced problem in the first place.

Her note to Vikram is short: *"Their model may be fine, but 91% accuracy tells us almost nothing on a 9.7% churn rate — predicting 'no churn' for everyone already gets 90.3%. Ask them for AUC and log loss against our own base rate, the way we evaluate every model here, and for what their training data actually is: a deep network is only as good as what it was trained on, and 'deep learning' is not by itself evidence of anything."* The vendor comes back with an AUC of 0.79 on a held-out sample — competitive with Riverstone's own gradient-boosting model, not clearly better than it, and considerably harder to explain to the sales team than either the boosted model or the logistic regression Chapter 37 already uses.

The point of this story isn't that deep learning is overhyped; this chapter's own honest numbers show it does real, useful things, on the right problems. The point is that "deep learning" is a *method*, not a *result*, and every method in this book — however sophisticated — still has to clear the same bar: beat the baseline, on your own data, measured honestly.

Tools

- **PyTorch 2.14.0** (`pip install torch`; CPU-only build works fine for everything in this chapter): `nn.Linear`, `nn.Conv2d`, `nn.ReLU`, `nn.Tanh`, `nn.Sigmoid`, `nn.Sequential`, `nn.BCELoss`, `nn.BCEWithLogitsLoss`, `nn.CrossEntropyLoss`, `torch.optim.SGD`, `torch.optim.Adam`, and `.backward()` for automatic differentiation.
 - **scikit-learn 1.8.0**: `load_digits` (a small, built-in image dataset — no download needed), plus the same pipeline, splitting, and evaluation tools used throughout Part IV.
 - Not used here but worth knowing: **torchvision** (real pretrained image models, needing an internet connection to download weights), **Keras/TensorFlow** (an alternative to PyTorch with a similar feature set), **Weights & Biases** or **TensorBoard** (tracking training runs, essential once experiments multiply).
 - Everything in this chapter ran on one CPU core, Python 3.12.3, on 18 September 2026, in well under a minute total.
 - **Companion files**: this chapter needs no new dataset; `companion/accounts/accounts.csv` (Chapter 37) and scikit-learn's bundled digits dataset are both already available. Run the chapter's code from `companion/ch43/`.
-

The project: a tabular network and an image classifier, evaluated honestly

Goal: hands-on practice building, training, and — most importantly — fairly evaluating two small neural networks, one on tabular data and one on images.

Steps:

1. **Neuron and XOR.** Reproduce section 43.2's single-neuron failure and two-layer success on XOR. Then design a slightly harder toy problem, by hand, that also can't be solved by a straight line, and confirm a hidden layer solves it.
2. **Forward pass by hand.** Pick your own tiny set of weights (2 inputs, 2 or 3 hidden neurons, 1 output) and compute the forward pass on paper before checking it in PyTorch.
3. **Gradient check.** Verify autograd's gradient against a hand-nudged numerical gradient for at least one weight, as section 43.4 did.
4. **Tabular network.** Train a small network on the Riverstone accounts data (or your own tabular dataset), and compare it honestly with logistic regression and gradient boosting using Chapter 39's metrics (AUC, log loss), not just accuracy.

5. **Image classifier.** Train a small CNN on the digits dataset (or another small image dataset), and report a proper held-out accuracy.
6. **Transfer learning.** Split your image classes into a "base" and "new" task as section 43.7 did, and test transfer learning against training from scratch at several training-set sizes. Report the honest result, whichever way it goes.
7. **Write a one-page recommendation:** for your tabular problem, which method would you actually deploy, and why? For your image problem, at what point (if any) does transfer learning pay off?

Stretch goals:

- Add **dropout** (`nn.Dropout`) to the tabular network and see whether it changes the validation AUC.
- Try a **deeper** network (more hidden layers) on the tabular data and check whether validation performance improves or gets worse — connect the result to Chapter 37's bias–variance discussion.
- Use `torchvision.datasets` (if internet access allows downloading) to try transfer learning from a model actually pretrained on natural images, and compare the strength of that transfer with section 43.7's small-base-model example.
- Plot the training and validation loss curves for the tabular network across all 200 epochs, and identify the point (if any) where it starts to overfit.

You've got it when...

- ☐ I can explain that a single neuron is exactly Chapter 37's logistic regression, with an activation function.
 - ☐ I can explain why one neuron cannot solve every classification problem, using XOR as a concrete example.
 - ☐ I can compute a small network's forward pass by hand and match it exactly in code.
 - ☐ I understand backpropagation as the chain rule, applied automatically, and I've verified it against a numerical gradient at least once.
 - ☐ I never assume a neural network beats gradient boosting on tabular data without testing it.
 - ☐ I can explain, in plain terms, what a convolution does to an image.
 - ☐ I evaluate an image classifier (or any model) with a proper held-out test, not training accuracy.
 - ☐ I know that transfer learning's value depends entirely on what the base model was trained on, and I check rather than assume it helps.
 - ☐ I judge every deep learning claim against the same baseline discipline as every other method in this book.
-

Recap

- A **neuron** is a weighted sum plus a bias, passed through an **activation function** — exactly Chapter 37's logistic regression when the activation is sigmoid.
 - **ReLU** avoids the vanishing-gradient problem of sigmoid in hidden layers by having a gradient of exactly 0 or 1.
 - A single neuron can only draw a straight decision boundary; problems like **XOR** need at least one **hidden layer** with a nonlinear activation to be solvable at all.
 - **Backpropagation** computes the exact gradient of every parameter via the chain rule, automated by libraries like PyTorch's `autograd`; it can be verified against a slow, hand-nudged numerical gradient.
 - On Riverstone's tabular churn data, a small neural network narrowly beats logistic regression and loses to tuned gradient boosting — deep learning has no automatic advantage on clean tabular data.
 - A **convolution** slides a learned kernel across an image to detect patterns; stacked convolutional layers build up from simple to complex features automatically.
 - **Transfer learning** reuses a base model's early layers on a new task; its benefit depends entirely on how rich and general the base model's training data was, not on the technique alone — check it, don't assume it.
 - Every deep learning claim should clear the same bar as any other method: beat an honest baseline, measured with the tools from Chapters 36, 37, and 39.
-

Practice exercises

Code exercises run from `companion/ch43/` after the chapter's code (they use `X_train`, `y_train`, `X_valid`, `y_valid`, `network`, `base_model`, `SmallCNN`, `small_training_set`, `Xn_test_t`, `yn_test_t`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) A neuron has weights $[0.4, -0.5]$, bias 0.2, and input $[3, 2]$. Compute z and then the sigmoid output.
2. (*hand*) Compute $\text{ReLU}(-3)$, $\text{ReLU}(0)$, and $\text{ReLU}(4.5)$.
3. (*hand*) Explain in one sentence why a single neuron cannot solve XOR, without using the word "linear."
4. A network has 2 inputs, one hidden layer of 5 neurons, and 1 output neuron, with biases at every neuron. How many total parameters does it have? (Show the weights and biases separately.)

Core

5. Retrain the single neuron on XOR with a much smaller learning rate (`lr=0.01`) for the same 2000 steps. Does it now solve XOR? What does that tell you about the difference between "not enough

training" and "cannot represent the pattern at all"?

6. Change the two-layer XOR network's hidden layer from 4 neurons to 2. Does it still solve XOR after 3000 steps? Try 1 hidden neuron. What's the smallest hidden layer that works?
7. Add a third hidden layer to the tabular network (`nn.Linear(8, 8)`, `nn.ReLU()` before the final layer) and retrain. Does validation AUC improve, get worse, or stay about the same? Relate your answer to Chapter 37's bias-variance discussion.
8. Train the tabular network for 500 epochs instead of 200, printing validation AUC every 100 epochs. Does it keep improving, or does it start to get worse partway through? What would that pattern mean?
9. Retrain the digit CNN (`base_model`) using only 200 training images instead of 675 (sample randomly). How much does test accuracy fall? What does this suggest about how much data a from-scratch CNN needs?
10. For the transfer-learning comparison, extend section 43.7's table to `n_per_class` values of 1 and 50. Does the pattern (transfer ahead at very small `n`, from-scratch ahead at larger `n`) continue at both extremes?

Stretch

11. Modify `train_transfer` to unfreeze `conv2` (but keep `conv1` frozen) and fine-tune it along with the classifier head. Does partial fine-tuning close the gap with training from scratch at `n_per_class=30` ?
12. Build a base CNN trained on digits 0–7 (8 classes) instead of 0–4 (5 classes), then transfer to classifying just 8 and 9. Does having a richer, more varied base task improve transfer learning's results compared with section 43.7?
13. Implement a tiny two-parameter linear model ($y = wx + b$) in raw PyTorch tensors with `requires_grad=True` , train it with a manual gradient descent loop (no `torch.optim`), and check that it recovers weights close to a `LinearRegression` fit on the same synthetic data.

Think about it

14. A colleague argues: "Deep learning is state of the art, so we should always use it." Using this chapter's own numbers, write a two-sentence response.
15. Explain to a non-technical stakeholder, in three sentences, why a hidden layer is necessary for a network to learn some patterns, using the XOR example without technical jargon.
16. A vendor says their transfer-learning-based product was "pretrained on millions of images" and should work well for Riverstone's specific product-defect photos. What one question from this chapter would you ask before trusting that claim?

Key terms

neuron · activation function · sigmoid · ReLU (rectified linear unit) · vanishing gradient · layer · hidden layer · neural network · weight · bias · forward pass · XOR problem · decision boundary · backpropagation · chain rule · autograd · numerical gradient check · `BCELoss` / `BCEWithLogitsLoss` · epoch · convolution · kernel (filter) · convolutional neural network (CNN) · pooling · feature map · transfer learning · base task · pretrained model · frozen layers · fine-tuning · from-scratch training

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 37, Supervised Learning Algorithms**, is this chapter's constant point of comparison: logistic regression and gradient boosting, both still very much in play.
 - **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, governs how any deep learning result should be judged, unchanged.
 - **Chapter 41, NLP Foundations**, previewed learned representations with word embeddings; Chapter 54 scales that idea up enormously.
 - **Chapter 53, Deep Learning in Practice** (later in the book), goes further into architectures, regularization, and training at scale.
 - **Chapter 54, Generative AI & Large Language Models**, is where transfer learning's real power shows up: models pretrained on vastly larger, richer data than anything in this chapter.
 - **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers backpropagation, activation functions, why XOR needs a hidden layer, and "when would you use deep learning versus a simpler model?" — one of the most common questions in applied ML interviews, and one this chapter now lets you answer with evidence.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. $z = 0.4 \times 3 + (-0.5) \times 2 + 0.2 = 1.2 - 1.0 + 0.2 = \mathbf{0.4}$. $\text{sigmoid}(0.4) = 1 \div (1 + e^{-0.4}) = 1 \div 1.6703 = \mathbf{0.599}$.
2. $\text{ReLU}(-3) = \mathbf{0}$ (negative input, zeroed). $\text{ReLU}(0) = \mathbf{0}$ (exactly at the boundary). $\text{ReLU}(4.5) = \mathbf{4.5}$ (positive input, passed through unchanged).
3. A single neuron computes one weighted sum and applies one activation, which is equivalent to drawing one straight (or, after the activation, one smoothly curved but still single, unbroken) dividing surface between the two output classes; XOR's four points can't be split into "0"s and "1"s by any such single dividing surface, however it's tilted or positioned, because the two "1" points sit diagonally opposite each other with a "0" point on each of the other diagonal corners.

4. From input to hidden layer: 2 inputs $\times$ 5 hidden neurons = 10 weights, plus 5 biases (one per hidden neuron) = 15 parameters. From hidden to output: 5 hidden neurons $\times$ 1 output = 5 weights, plus 1 bias = 6 parameters. **Total: 15 + 6 = 21 parameters.**

5.

```
torch.manual_seed(43)
slow_neuron = nn.Sequential(nn.Linear(2, 1), nn.Sigmoid())
slow_optimizer = torch.optim.SGD(slow_neuron.parameters(), lr=0.01)
for step in range(2000):
    slow_optimizer.zero_grad()
    slow_pred = slow_neuron(X_xor)
    slow_loss = loss_fn(slow_pred, y_xor)
    slow_loss.backward()
    slow_optimizer.step()
print(
    f"loss after 2000 steps at lr=0.01: {slow_loss.item():.4f}  "
    f"predictions: {slow_pred.detach().numpy().round(3).ravel()}"
)
```

```
loss after 2000 steps at lr=0.01: 0.6951  predictions: [0.548 0.494 0.521 0.467]
```

At this much smaller learning rate, the loss (0.6951) is barely different from the $\ln(2) \approx 0.693$ "guessing" baseline, and the predictions (around 0.5 for every input) show the neuron still hasn't found any useful pattern — but this time it's genuinely still in the early stages of training, not stuck at a true floor the way the original run at `lr=0.5` was. The key difference from section 43.2: here, more training steps or a higher learning rate really would help eventually approach a slightly-better-than-guessing compromise (since a single neuron can partially exploit any imbalance, though not solve XOR outright), whereas the original example had already converged and *couldn't* do better no matter how long it ran. Comparing a "stuck and converged" loss curve against a "still moving, just slowly" one is exactly how you tell insufficient training apart from a genuine representational limit.

6.

```
for hidden_size in [2, 1]:
    torch.manual_seed(43)
    small_net = nn.Sequential(
        nn.Linear(2, hidden_size), nn.Tanh(), nn.Linear(hidden_size, 1), nn.Sigmoid()
    )
    small_optimizer = torch.optim.SGD(small_net.parameters(), lr=0.5)
    for step in range(3000):
        small_optimizer.zero_grad()
        small_pred = small_net(X_xor)
        small_loss = loss_fn(small_pred, y_xor)
        small_loss.backward()
        small_optimizer.step()
    print(
        f"{hidden_size} hidden neuron(s): loss {small_loss.item():.4f}  "
        f"predictions {small_pred.detach().numpy().round(3).ravel()}"
    )
```

```
2 hidden neuron(s): loss 0.0022  predictions [0.002 0.997 0.997 0.001]
1 hidden neuron(s): loss 0.4799  predictions [0.668 0.665 0.665 0.004]
```

With **2** hidden neurons, the network still solves XOR essentially perfectly (loss 0.0022, predictions [0.002, 0.997, 0.997, 0.001]). With only **1** hidden neuron, it fails (loss 0.480, predictions clustered around 0.665–0.668 for three of the four points): one hidden neuron is itself just a single straight-line boundary feeding into the output, so it inherits the same structural limitation as section 43.2's single neuron. **Two is the smallest hidden layer that solves XOR** — matching the geometric intuition that you need at least two lines to carve out the diagonal pattern XOR requires.

7.

```
torch.manual_seed(43)
deeper_network = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
deeper_optimizer = torch.optim.Adam(deeper_network.parameters(), lr=0.01)
for epoch in range(200):
    deeper_optimizer.zero_grad()
    deeper_loss = loss_fn3(deeper_network(Xtr_t), ytr_t)
    deeper_loss.backward()
    deeper_optimizer.step()
with torch.no_grad():
    deeper_pred = torch.sigmoid(deeper_network(Xva_t)).numpy().ravel()
print(
    f"3 hidden layers: AUC {roc_auc_score(y_valid, deeper_pred):.3f}  "
    f"(2 hidden layers, chapter version: {roc_auc_score(y_valid, final_pred):.3f})"
)
```

```
3 hidden layers: AUC 0.770   (2 hidden layers, chapter version: 0.773)
```

Adding a third hidden layer makes validation AUC slightly *worse* (0.770 against the two-layer version's 0.773), not better. With only 3,000 training rows and 25 features, the extra layer adds capacity the data doesn't need and doesn't reward, exactly Chapter 37's bias–variance lesson: past a certain point, more model flexibility increases variance without reducing bias, and the honest way to find that point is to test it, not to assume "deeper is better."

8.

```

torch.manual_seed(43)
long_network = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
long_optimizer = torch.optim.Adam(long_network.parameters(), lr=0.01)
for epoch in range(500):
    long_optimizer.zero_grad()
    long_loss = loss_fn3(long_network(Xtr_t), ytr_t)
    long_loss.backward()
    long_optimizer.step()
    if epoch % 100 == 0:
        with torch.no_grad():
            long_pred = torch.sigmoid(long_network(Xva_t)).numpy().ravel()
        print(
            f"epoch {epoch:>3}: training loss {long_loss.item():.4f}  "
            f"validation AUC {roc_auc_score(y_valid, long_pred):.3f}"
        )

```

```

epoch  0: training loss 0.6994   validation AUC 0.542
epoch 100: training loss 0.2373   validation AUC 0.762
epoch 200: training loss 0.2185   validation AUC 0.774
epoch 300: training loss 0.1922   validation AUC 0.760
epoch 400: training loss 0.1607   validation AUC 0.739

```

Validation AUC rises to a peak of 0.774 around epoch 200, then **declines** to 0.739 by epoch 400, even as the training loss keeps falling the whole time (0.699 → 0.161). That divergence — training loss still improving while validation performance gets worse — is the textbook signature of **overfitting**: the network is increasingly memorizing quirks of the 3,000 training rows rather than learning patterns that generalize. In practice this is managed with **early stopping** (stop training once validation performance stops improving) rather than picking a fixed epoch count in advance.

9.

```

small_indices = np.random.default_rng(43).choice(len(Xb_train), 200, replace=False)
Xb_small_t, yb_small_t = to_tensors(Xb_train[small_indices], yb_train[small_indices])
torch.manual_seed(43)
small_data_model = SmallCNN(n_classes=5)
small_optimizer = torch.optim.Adam(small_data_model.parameters(), lr=0.01)
for epoch in range(60):
    small_optimizer.zero_grad()
    small_loss = loss_fn4(small_data_model(Xb_small_t), yb_small_t)
    small_loss.backward()
    small_optimizer.step()
with torch.no_grad():
    small_accuracy = (
        (small_data_model(Xb_test_t).argmax(1) == yb_test_t).float().mean().item()
    )
print(
    f"200 training images: test accuracy {small_accuracy:.1%}  "
    f"(675 training images, chapter version: {base_accuracy:.1%})"
)

```

```

200 training images: test accuracy 96.9%   (675 training images, chapter version: 98.7%)

```

Test accuracy falls only modestly, from 98.7% with 675 images to 96.9% with 200 — a reminder that this is a genuinely easy task (small, clean, centered 8×8 digit images with only 5 classes), so even a substantial cut in training data costs relatively little. On harder, more realistic image problems (more classes, more visual variation, more noise), a comparable cut in training data typically costs far more, which is exactly why transfer learning and data augmentation matter more in practice than this particular toy comparison suggests.

10.

```
for n in [1, 50]:
    transfer_acc = train_transfer(n)
    scratch_acc = train_scratch(n)
    print(f"{n:>15}    {transfer_acc:>27.1%}    {scratch_acc:>13.1%}")
```

1	58.9%	59.8%
50	95.1%	98.2%

The pattern from section 43.7 (from-scratch pulling ahead as data grows) holds cleanly at **n=50**: 98.2% against transfer's 95.1%, the largest gap in favor of training from scratch across the whole sweep. At the opposite extreme, **n=1** (one example per class, five images total) doesn't show the clean reversal you might expect: the two methods are essentially tied (58.9% transfer against 59.8% from scratch), both well below their performance at every larger sample size. With a single example per class, both approaches are close to memorizing one exemplar per digit rather than learning anything general, and the result is noisy rather than a reliable signal either way. The honest lesson from the full sweep, n=1 through 50: **transfer learning's advantage is real but not guaranteed at every level of data scarcity** — it shows up clearly in the middle of the range (a handful to a few dozen examples per class) and gets noisier, not more clearly favorable, at the very extreme of almost no data at all.

11.

```
def train_transfer_partial(n_per_class, epochs=80):
    Xs, ys = small_training_set(n_per_class)
    model = copy.deepcopy(base_model)
    for param in model.conv1.parameters():
        param.requires_grad = False # conv1 stays frozen
    model.fc1 = nn.Linear(16 * 2 * 2, 32)
    model.fc2 = nn.Linear(32, 5)
    trainable = filter(lambda p: p.requires_grad, model.parameters())
    optimizer = torch.optim.Adam(trainable, lr=0.01)
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xn_test_t).argmax(1) == yn_test_t).float().mean().item()

partial_acc = train_transfer_partial(30)
print(
    f"n=30, conv2 unfrozen: {partial_acc:.1%}    "
    f"(fully frozen: 92.9%, from scratch: 98.2%)"
)
```

n=30, conv2 unfrozen: 97.8% (fully frozen: 92.9%, from scratch: 98.2%)

Unfreezing `conv2` and letting it adapt alongside the classifier head (97.8%) closes most of the gap between fully frozen features (92.9%) and training from scratch (98.2%), without quite matching training everything from nothing. Partial fine-tuning is a middle ground, letting the later, more task-specific convolutional features adjust while the earliest, most general edge-detecting features stay fixed. This is standard practice in real transfer learning: freeze early layers, fine-tune later ones, and how many layers to unfreeze is itself a choice to test, not a fixed rule.

12.

```

richer_mask = labels < 8 # digits 0-7 as the base task
X_rich, y_rich = images[richer_mask], labels[richer_mask]
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    X_rich, y_rich, test_size=0.25, random_state=43, stratify=y_rich
)
Xr_train_t, yr_train_t = to_tensors(Xr_train, yr_train)

torch.manual_seed(43)
richer_base = SmallCNN(n_classes=8)
richer_optimizer = torch.optim.Adam(richer_base.parameters(), lr=0.01)
for epoch in range(60):
    richer_optimizer.zero_grad()
    loss = loss_fn4(richer_base(Xr_train_t), yr_train_t)
    loss.backward()
    richer_optimizer.step()

final_mask = (labels == 8) | (labels == 9)
X_final, y_final = images[final_mask], labels[final_mask] - 8
Xf_train, Xf_test, yf_train, yf_test = train_test_split(
    X_final, y_final, test_size=0.25, random_state=43, stratify=y_final
)
Xf_test_t, yf_test_t = to_tensors(Xf_test, yf_test)

def transfer_from_richer_base(n_per_class, epochs=80):
    chosen = []
    for digit_class in [0, 1]:
        candidates = np.where(yf_train == digit_class)[0]
        chosen.extend(
            np.random.default_rng(43).choice(candidates, n_per_class, replace=False)
        )
    Xs, ys = to_tensors(Xf_train[chosen], yf_train[chosen])
    model = copy.deepcopy(richer_base)
    for param in model.conv1.parameters():
        param.requires_grad = False
    for param in model.conv2.parameters():
        param.requires_grad = False
    model.fc1 = nn.Linear(16 * 2 * 2, 32)
    model.fc2 = nn.Linear(32, 2)
    optimizer = torch.optim.Adam(
        filter(lambda p: p.requires_grad, model.parameters()), lr=0.01
    )
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xf_test_t).argmax(1) == yf_test_t).float().mean().item()

print(
    f"transfer from an 8-class base (digits 0-7) to a 2-class task (8 vs 9), 10/class: "
    f"{transfer_from_richer_base(10):.1%}"
)

```

```

transfer from an 8-class base (digits 0-7) to a 2-class task (8 vs 9), 10/class: 93.3%

```

Transfer from the richer, 8-class base reaches 93.3% on the new 8-vs-9 task with only 10 examples per class — a strong result for a two-class problem with so little new-task data. The general principle behind why this can work at all is the same one behind real-world transfer learning: a base model

exposed to more classes and more total images during its own training tends to learn broader, more reusable early-layer features (edges, strokes, curves) than one trained on a narrower slice of the same kind of data. This toy comparison can't isolate that effect cleanly (an 8-vs-9 task and a 5-class-base task aren't directly comparable numbers), but the direction matches what section 43.7 already showed: what the base model actually learned, not the technique by itself, is what decides whether transfer learning pays off.

13.

```
torch.manual_seed(43)
x_data = torch.linspace(0, 10, 50).unsqueeze(1)
y_data = (
    3.0 * x_data + 7.0 + torch.randn(50, 1) * 0.5
) # true relationship: y = 3x + 7, plus noise

w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
learning_rate = 0.01

for step in range(500):
    prediction = x_data * w + b
    loss = ((prediction - y_data) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad
        w.grad.zero_()
        b.grad.zero_()

print(f"manual gradient descent: w={w.item():.3f}    b={b.item():.3f}")

from sklearn.linear_model import LinearRegression

sklearn_fit = LinearRegression().fit(x_data.numpy(), y_data.numpy())
print(
    f"scikit-learn LinearRegression: w={sklearn_fit.coef_[0]:.3f}"
    f"    b={sklearn_fit.intercept_[0]:.3f}"
)
```

```
manual gradient descent: w=3.102    b=6.382
scikit-learn LinearRegression: w=3.027    b=6.885
```

The manually written gradient descent loop ($w=3.102$, $b=6.382$) lands close to both the true generating values ($w=3$, $b=7$, before noise) and scikit-learn's exact least-squares solution ($w=3.027$, $b=6.885$) — close but not identical, because 500 steps of gradient descent at this learning rate hasn't fully converged to the exact minimum the way scikit-learn's direct solution does, the same gap Chapter 37 noted between an iterative and an exact solver. What matters for this chapter is the equivalence it confirms: a "neural network" with no hidden layer and no activation function is just linear regression trained by gradient descent instead of solved directly — precisely Chapter 37's linear regression, viewed through the PyTorch machinery this chapter introduces. `.grad.zero_()` matters here exactly as `optimizer.zero_grad()` did throughout the chapter: without it, gradients from each step would keep accumulating on top of the previous ones.

- 14.** "This chapter's own numbers say otherwise on tabular data: a small neural network scored an AUC of 0.773 on our churn data, barely ahead of plain logistic regression (0.764) and clearly behind tuned gradient boosting (0.788), while needing far more code and tuning than either. 'State of the art' is true for images, text, and audio at scale — it is not automatically true for a table of account features, and the only way to know which situation you're in is to test it, the same way we test everything else."
- 15.** "Imagine trying to sort four boxes into two piles using only one straight cut of a knife — some patterns of boxes simply can't be separated that way, no matter where you make the cut. A single artificial 'neuron' can only make one such straight cut. Adding a hidden layer is like being allowed to make two cuts and then combine the results, which lets the network handle patterns that no single straight cut ever could."
- 16.** "What was the base model actually trained on, and how similar is that data to product-defect photos?" This chapter's own experiment showed transfer learning's value depends entirely on how rich and relevant the base model's training data is — a base model trained on millions of *generic* photographs (cats, cars, landscapes) may transfer only partially to a specific industrial-defect task, and the only way to know is to test the vendor's actual model on a genuine held-out sample of Riverstone's own defect photos, exactly as Chapter 39 would insist for any other vendor model.